

《移动终端软件开发技术》

Mobile Application Development

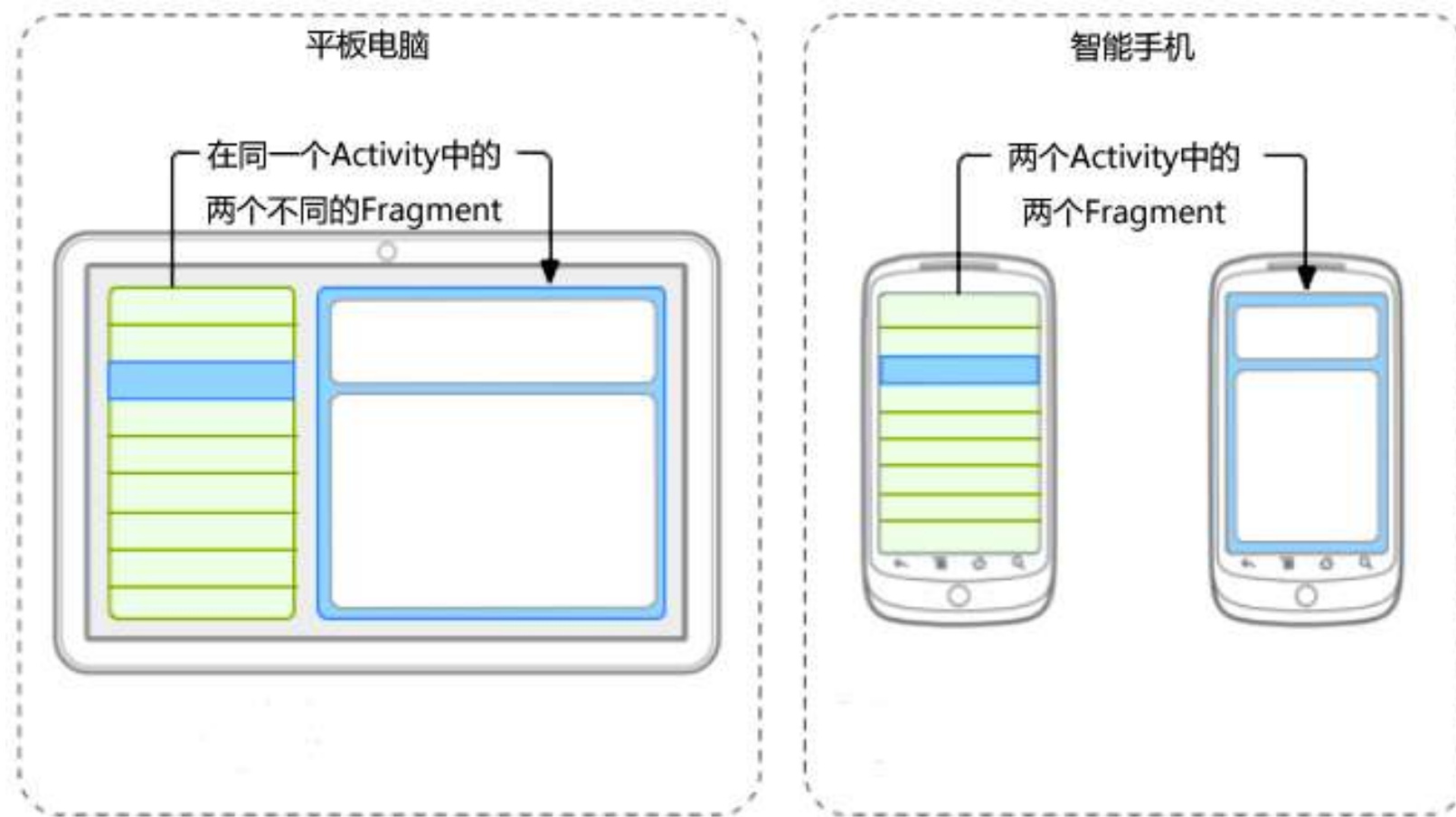
3.5 Android Fragment

谢坤
计算机学院（国家示范性软件学院）

什么是Fragment

- Fragment：可嵌入在Activity中的UI片段，能让程序更加合理和充分地利用大屏幕的空间
 - Activity片段：包含布局，拥有自己的生命周期
 - 嵌套于Activity，不能单独使用，受宿主Activity生命周期影响
 - 可被设计为可重用模块：可将同一个Fragment放置到多个不同的Activity中
 - 应避免直接从一个Fragment去操纵另一个Fragment：会增加两个Fragment之间的耦合度，不利于模块的重用

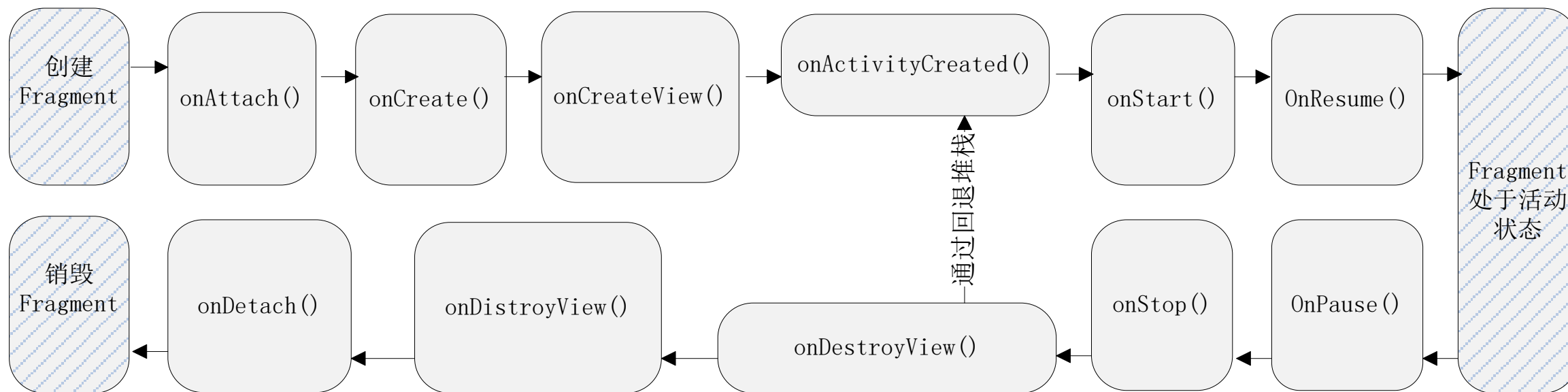
什么是Fragment



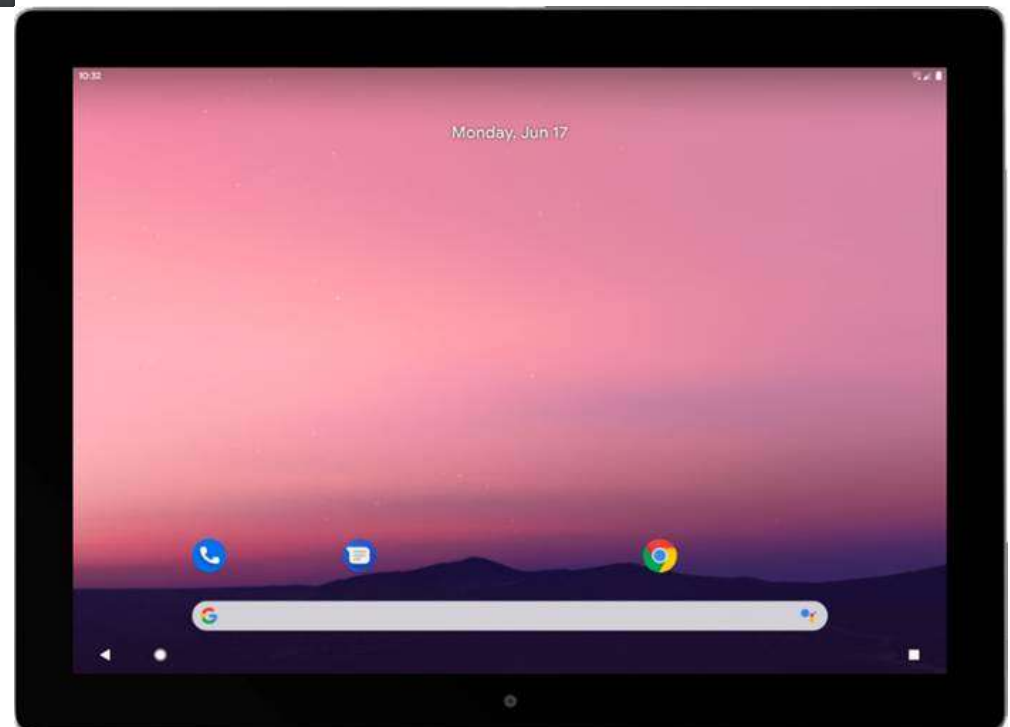
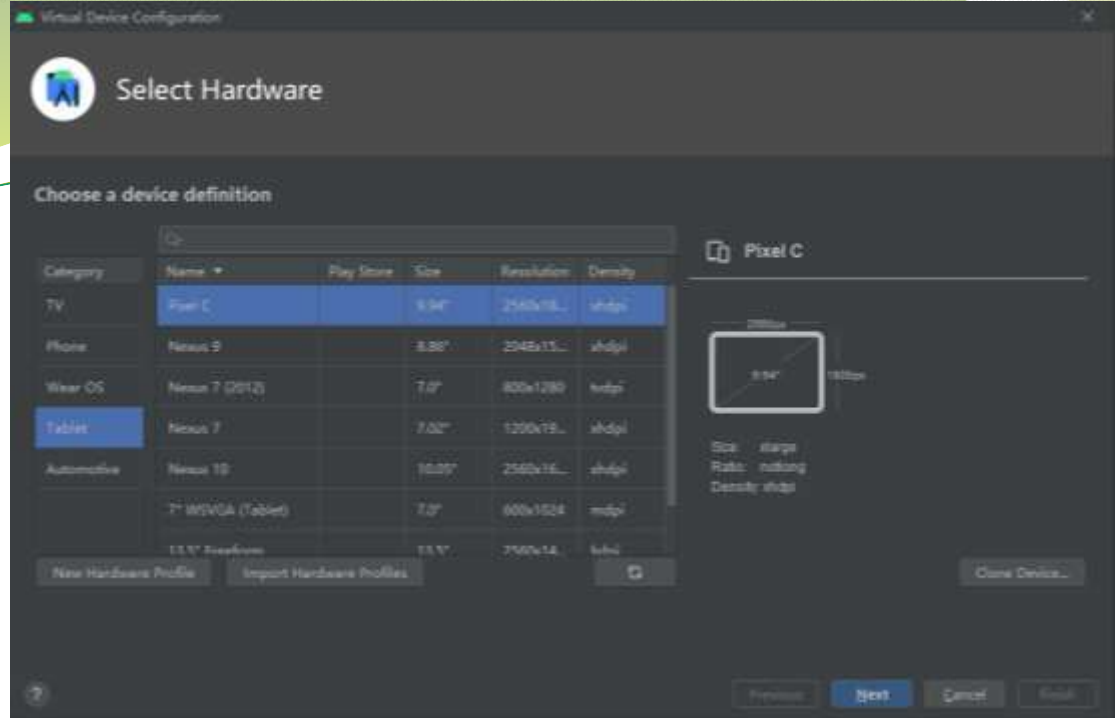
新闻阅读程序

什么是Fragment

- Fragment: 比Activity支持更多的事件回调函数。



Fragment的基本用法



静态加载Fragment

定义Fragment的布局:

- 左侧Fragment的布局left_fragment.xml
- 右侧Fragment的布局right_fragment.xml

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
```

```
<Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center_horizontal"
    android:text="Button"
/>
```

```
</LinearLayout>
```

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:background="#00ff00"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
```

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center_horizontal"
    android:textSize="24sp"
    android:text="This is right fragment"
/>
```

```
</LinearLayout>
```

静态加载Fragment

创建Fragment:

- LeftFragment类
- RightFragment类

```
class LeftFragment : Fragment() {  
  
    override fun onCreateView(inflater: LayoutInflater,  
                               container: ViewGroup?,  
                               savedInstanceState: Bundle?): View? {  
        return inflater.inflate(R.layout.left_fragment, container, false)  
    }  
  
}
```

```
class RightFragment : Fragment() {  
  
    override fun onCreateView(inflater: LayoutInflater,  
                               container: ViewGroup?,  
                               savedInstanceState: Bundle?): View? {  
        return inflater.inflate(R.layout.right_fragment, container, false)  
    }  
  
}
```

静态加载Fragment

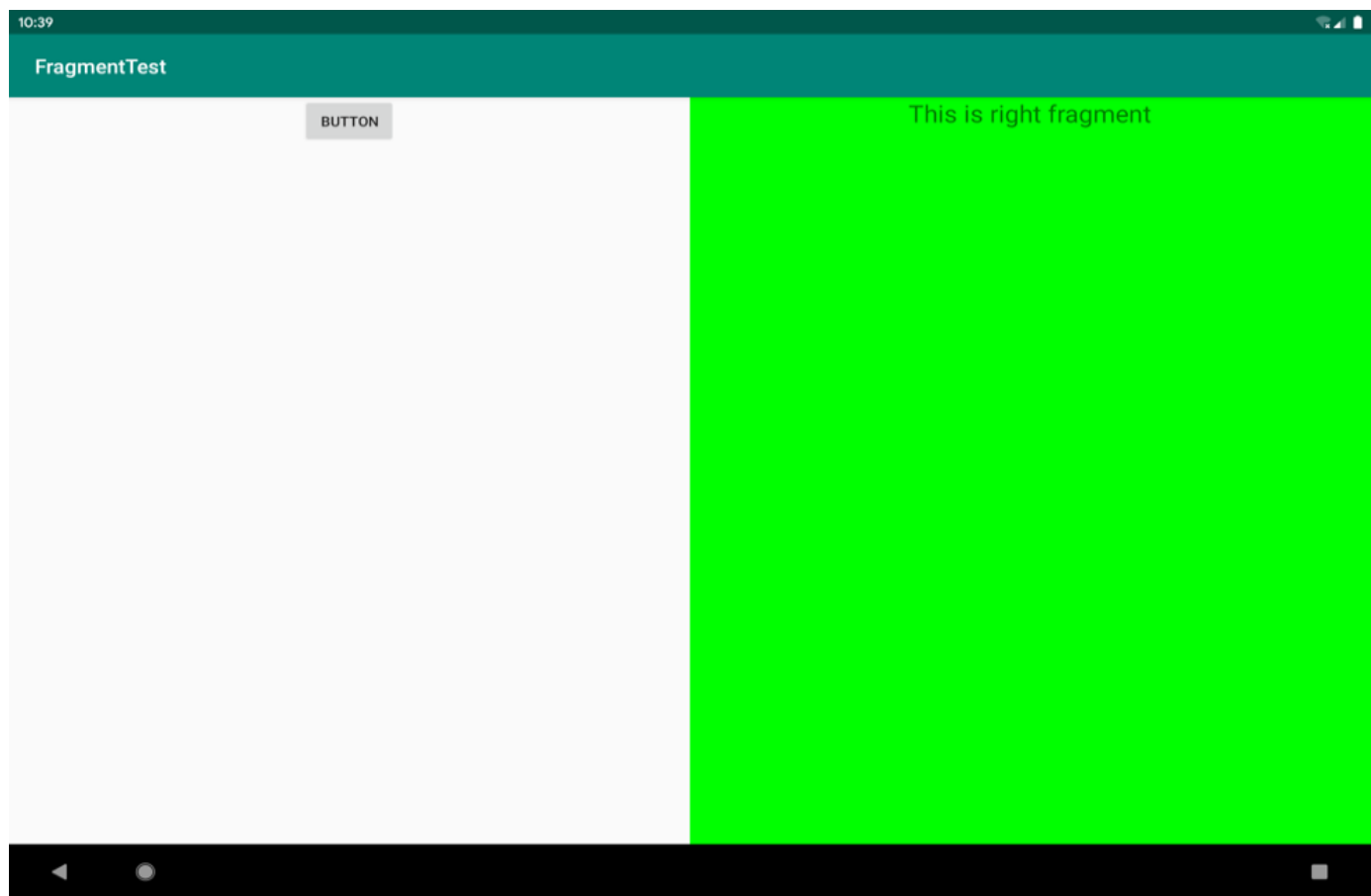
在布局中引入Fragment: activity_main.xml

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="match_parent" >

    <fragment
        android:id="@+id/leftFrag"
        android:name="com.example.fragmenttest.LeftFragment"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="1" />

    <fragment
        android:id="@+id/rightFrag"
        android:name="com.example.fragmenttest.RightFragment"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="1" />

</LinearLayout>
```



静态加载Fragment

静态加载Fragment的流程



动态加载Fragment

定义另一个Fragment布局并创建Fragment类:

- another_right_fragment.xml
- AnotherRightFragment类

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:background="#ffff00"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center_horizontal"
        android:textSize="24sp"
        android:text="This is another right fragment"
    />

</LinearLayout>
```

```
class AnotherRightFragment : Fragment() {

    override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?): View? {
        return inflater.inflate(R.layout.another_right_fragment, container, false)
    }

}
```

动态加载Fragment

调整Activity布局，添加FrameLayout，使用replace() 动态添加Fragment

```
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="match_parent" >

    <fragment
        android:id="@+id/leftFrag"
        android:name="com.example.fragmenttest.LeftFragment"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="1" />

    <FrameLayout
        android:id="@+id/rightLayout"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="1" >
    </FrameLayout>

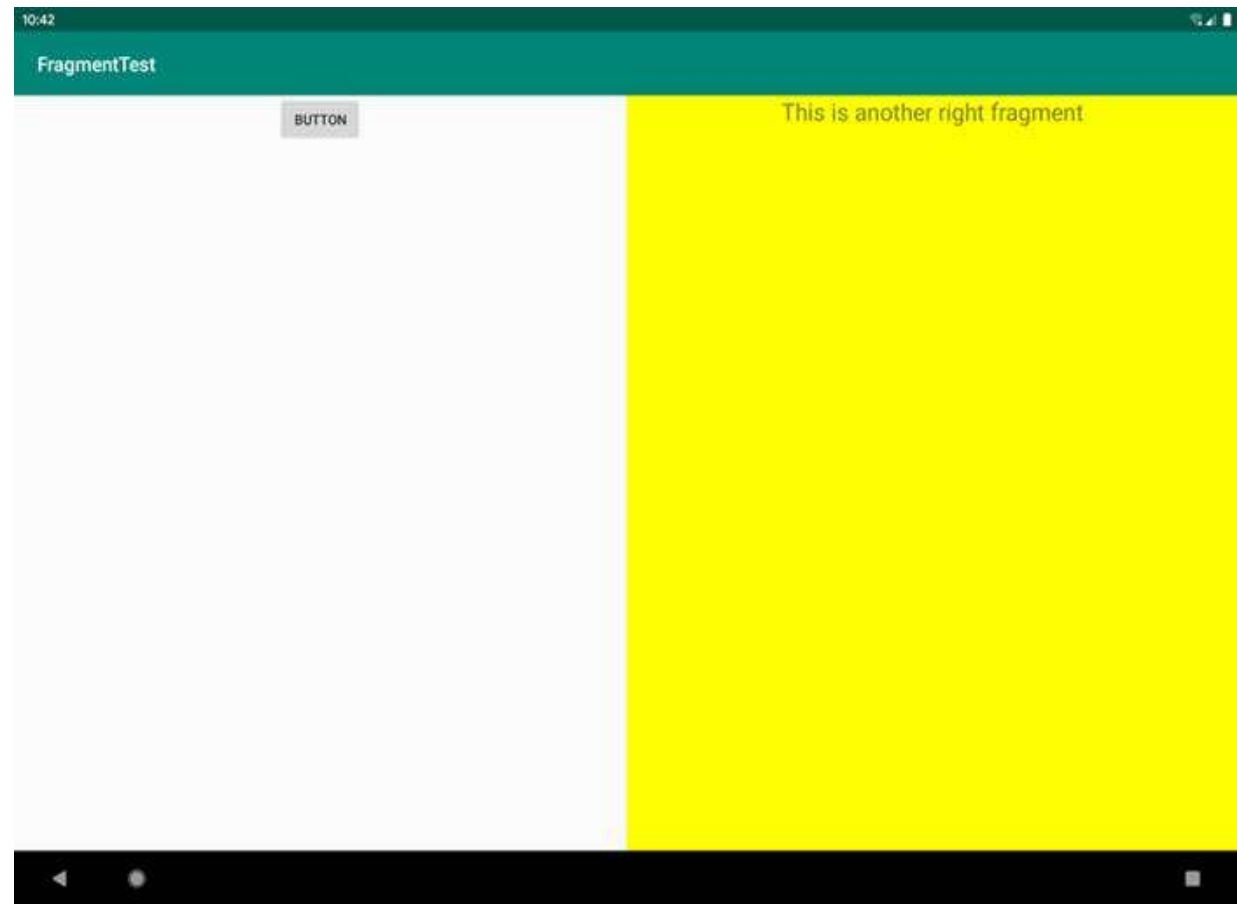
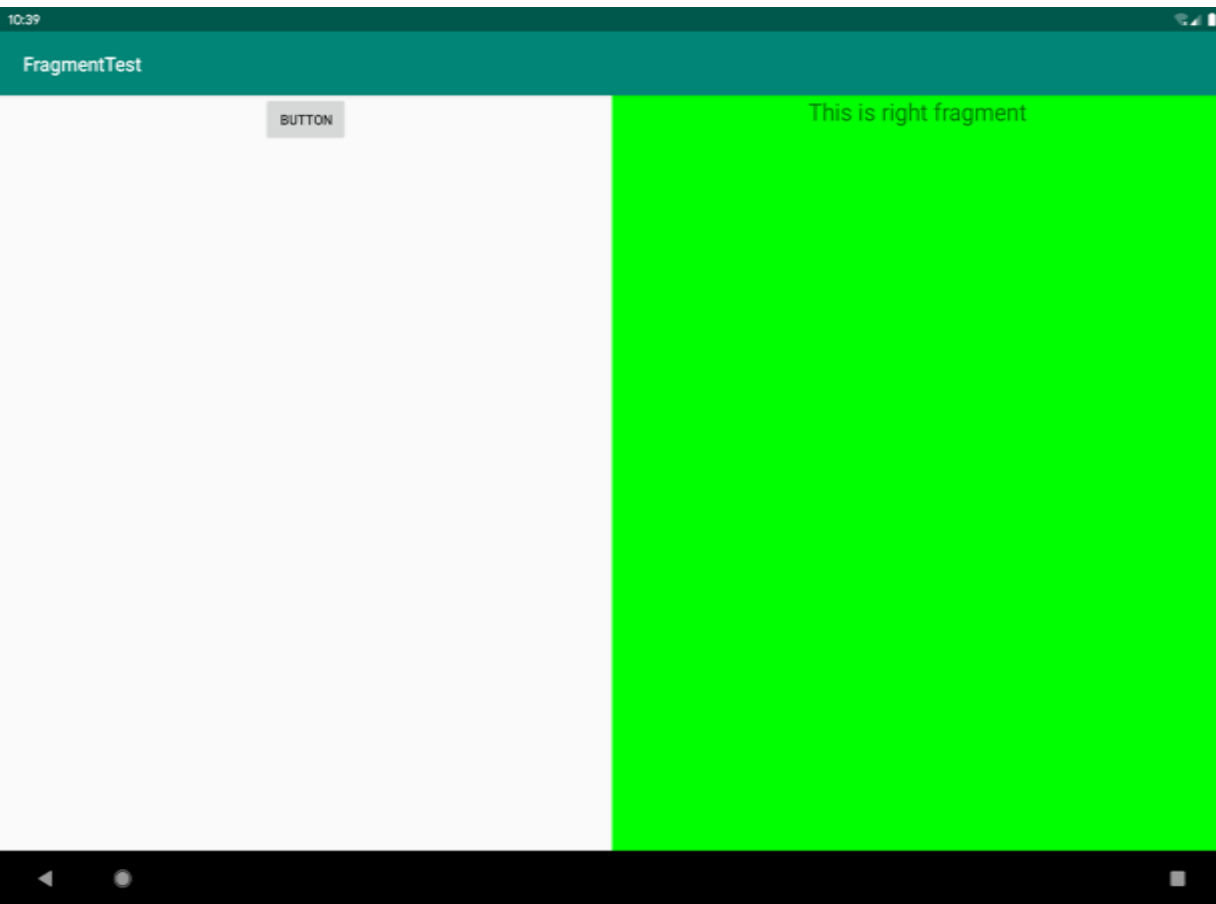
</LinearLayout>
```

```
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        button.setOnClickListener {
            replaceFragment(AnotherRightFragment())
        }
        replaceFragment(RightFragment())
    }

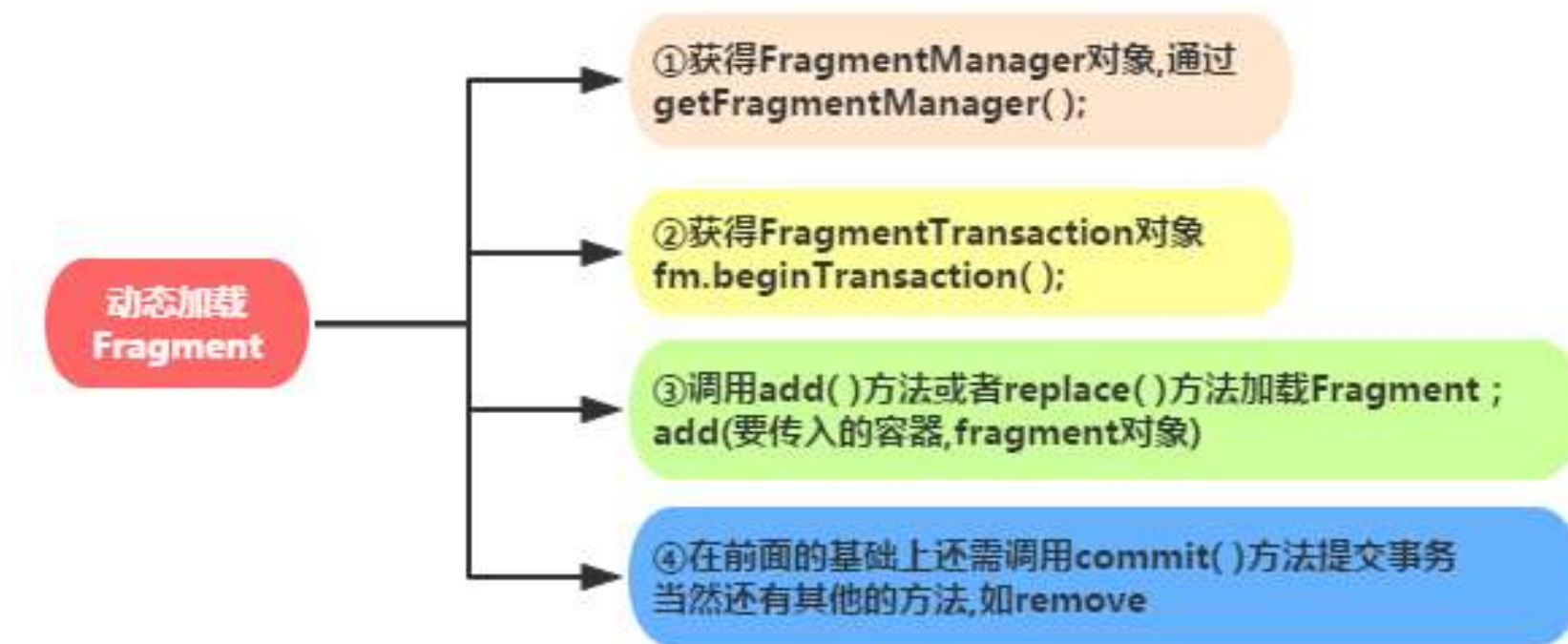
    private fun replaceFragment(fragment: Fragment) {
        val fragmentManager = supportFragmentManager
        val transaction = fragmentManager.beginTransaction()
        transaction.replace(R.id.rightLayout, fragment)
        transaction.commit()
    }
}
```

动态添加Fragment



动态加载Fragment

动态加载Fragment的流程



```
private fun replaceFragment(fragment: Fragment) {  
    val fragmentManager = supportFragmentManager  
    val transaction = fragmentManager.beginTransaction()  
    transaction.replace(R.id.rightLayout, fragment)  
    transaction.commit()  
}
```

Fragment实现返回栈

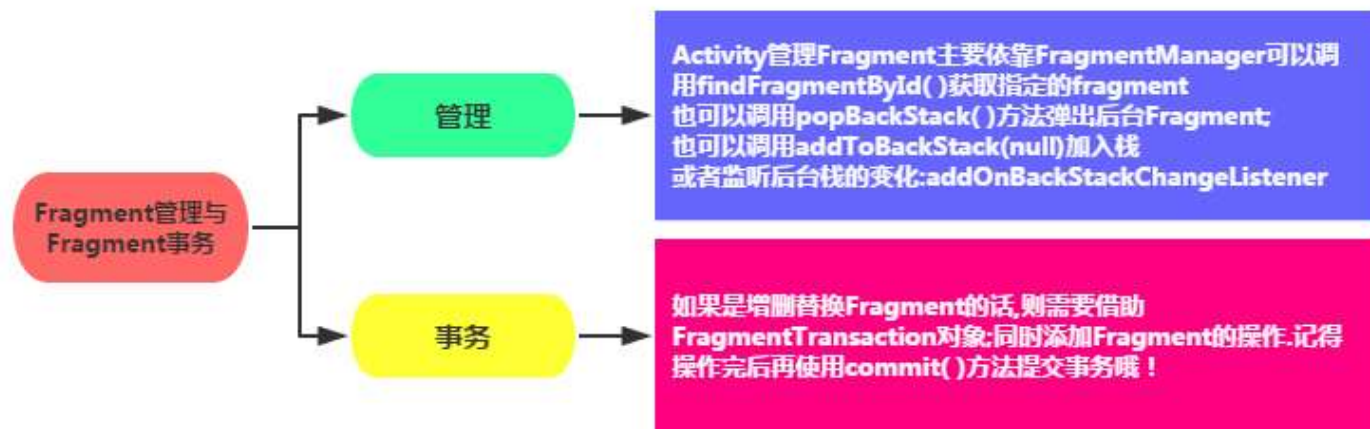
FragmentManager类:

- addToBackStack() 方法

```
class MainActivity : AppCompatActivity() {
```

```
...
```

```
private fun replaceFragment(fragment: Fragment) {  
    val fragmentManager = supportFragmentManager  
    val transaction = fragmentManager.beginTransaction()  
    transaction.replace(R.id.rightLayout, fragment)  
    transaction.addToBackStack(null)  
    transaction.commit()  
}  
}
```



Fragment与Activity的交互

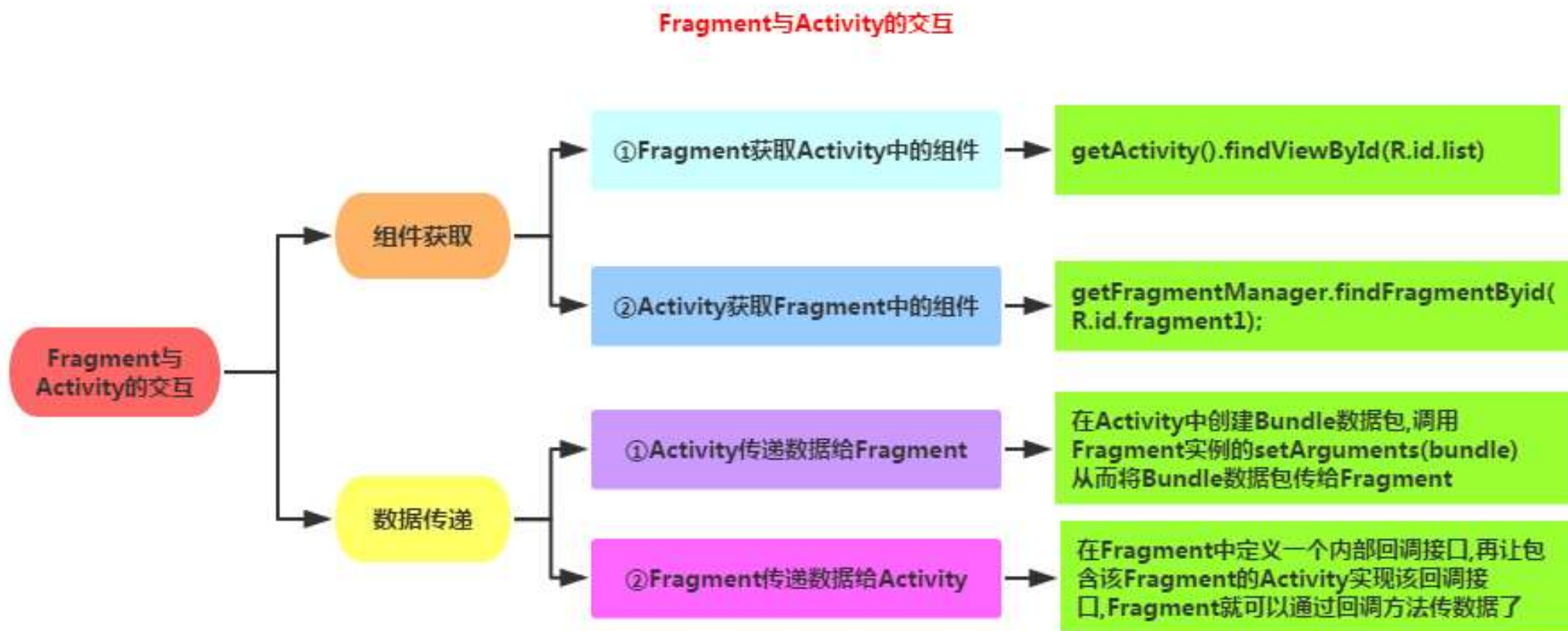
findFragmentById():

```
val fragment = supportFragmentManager.findFragmentById(R.id.leftFrag) as LeftFragment
```

```
val fragment = leftFrag as LeftFragment
```

```
if (activity != null) {  
    val mainActivity = activity as MainActivity  
}
```

Fragment与Activity的交互



ps:标识Fragment除了在布局文件中添加id属性标志,还可以使用Tag属性对Fragment进行标识,那么我们可以使用`findFragmentByTag`找到对应的Fragment了

Fragment的生命周期

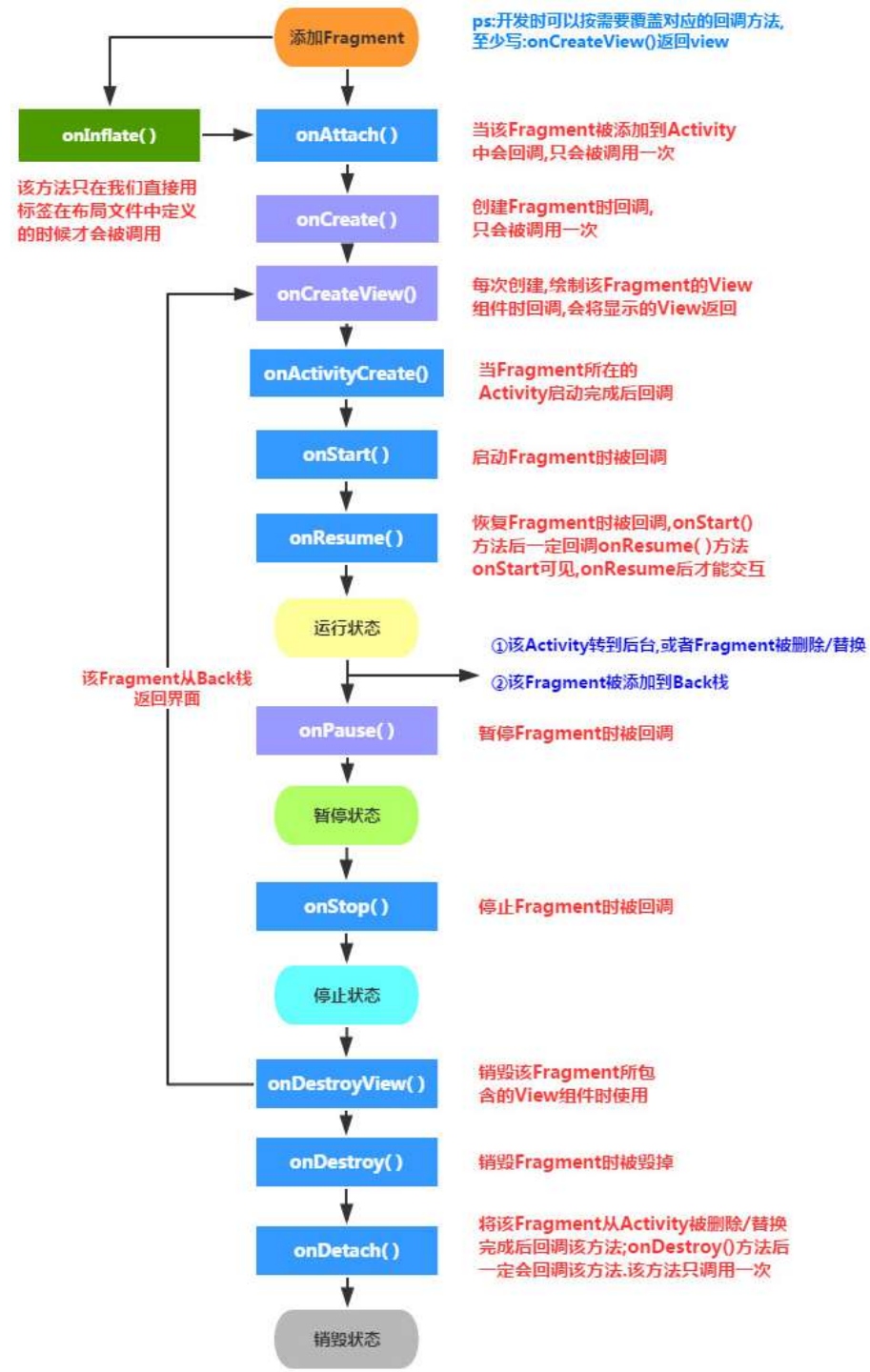
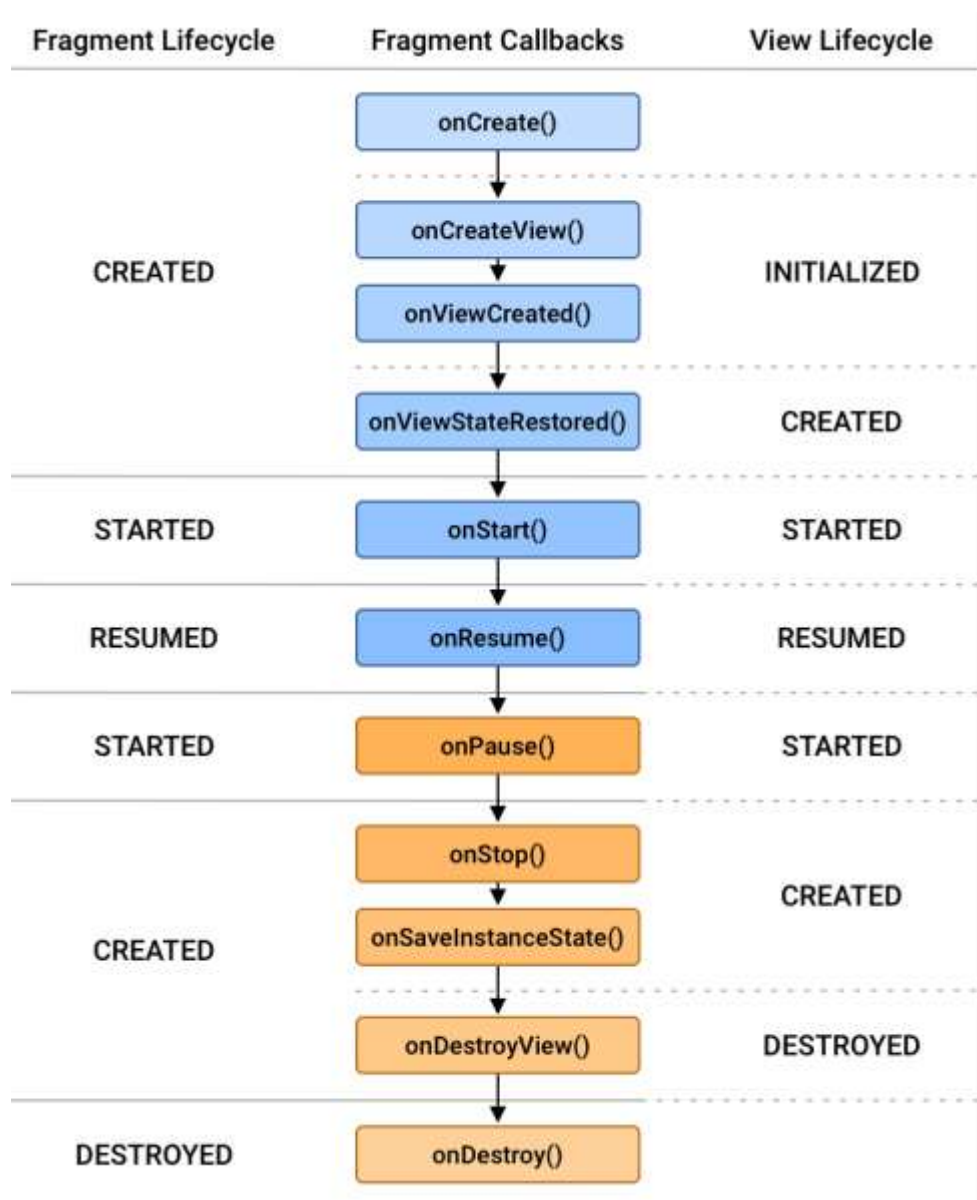
Fragment的状态和回调

- Fragment状态：
 - 运行状态
 - 暂停状态
 - 停止状态
 - 销毁状态

Fragment的状态和回调

- Fragment回调方法:
 - `onAttach()`
 - `onCreateView()`
 - `onActivityCreated()`
 - `onDestroyView()`
 - `onDetach()`

Fragment的生命周期示意图



Fragment生命周期示例

```
class RightFragment : Fragment() {
    companion object {
        const val TAG = "RightFragment"
    }
    override fun onAttach(context: Context) {
        super.onAttach(context)
        Log.d(TAG, "onAttach")
    }
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG, "onCreate")
    }
    override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?): View? {
        Log.d(TAG, "onCreateView")
        return inflater.inflate(R.layout.right_fragment, container, false)
    }
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        Log.d(TAG, "onViewCreated")
    }
    override fun onStart() {
        super.onStart()
        Log.d(TAG, "onStart")
    }
}
```

```
override fun onResume() {
    super.onResume()
    Log.d(TAG, "onResume")
}
override fun onPause() {
    super.onPause()
    Log.d(TAG, "onPause")
}
override fun onStop() {
    super.onStop()
    Log.d(TAG, "onStop")
}
override fun onDestroyView() {
    super.onDestroyView()
    Log.d(TAG, "onDestroyView")
}
override fun onDestroy() {
    super.onDestroy()
    Log.d(TAG, "onDestroy")
}
override fun onDetach() {
    super.onDetach()
    Log.d(TAG, "onDetach")
}
}
```

Fragment生命周期示例

```
com.example.fragmenttest (13338) ▼ Debug ▼ Q- RightFragment
13338/com.example.fragmenttest D/RightFragment: onAttach
13338/com.example.fragmenttest D/RightFragment: onCreate
13338/com.example.fragmenttest D/RightFragment: onCreateView
13338/com.example.fragmenttest D/RightFragment: onViewCreated
13338/com.example.fragmenttest D/RightFragment: onStart
13338/com.example.fragmenttest D/RightFragment: onResume
```

1

```
com.example.fragmenttest (13338) ▼ Debug ▼ Q- RightFragment
13338/com.example.fragmenttest D/RightFragment: onCreateView
13338/com.example.fragmenttest D/RightFragment: onStart
13338/com.example.fragmenttest D/RightFragment: onResume
13338/com.example.fragmenttest D/RightFragment: onPause
13338/com.example.fragmenttest D/RightFragment: onStop
13338/com.example.fragmenttest D/RightFragment: onDestroyView
```

2

```
com.example.fragmenttest (13338) ▼ Debug ▼ Q- RightFragment
13338/com.example.fragmenttest D/RightFragment: onStop
13338/com.example.fragmenttest D/RightFragment: onDestroyView
13338/com.example.fragmenttest D/RightFragment: onCreateView
13338/com.example.fragmenttest D/RightFragment: onViewCreated
13338/com.example.fragmenttest D/RightFragment: onStart
13338/com.example.fragmenttest D/RightFragment: onResume
```

3

```
com.example.fragmenttest (13338) ▼ Debug ▼ Q- RightFragment
13338/com.example.fragmenttest D/RightFragment: onResume
13338/com.example.fragmenttest D/RightFragment: onPause
13338/com.example.fragmenttest D/RightFragment: onStop
13338/com.example.fragmenttest D/RightFragment: onDestroyView
13338/com.example.fragmenttest D/RightFragment: onDestroy
13338/com.example.fragmenttest D/RightFragment: onDetach
```

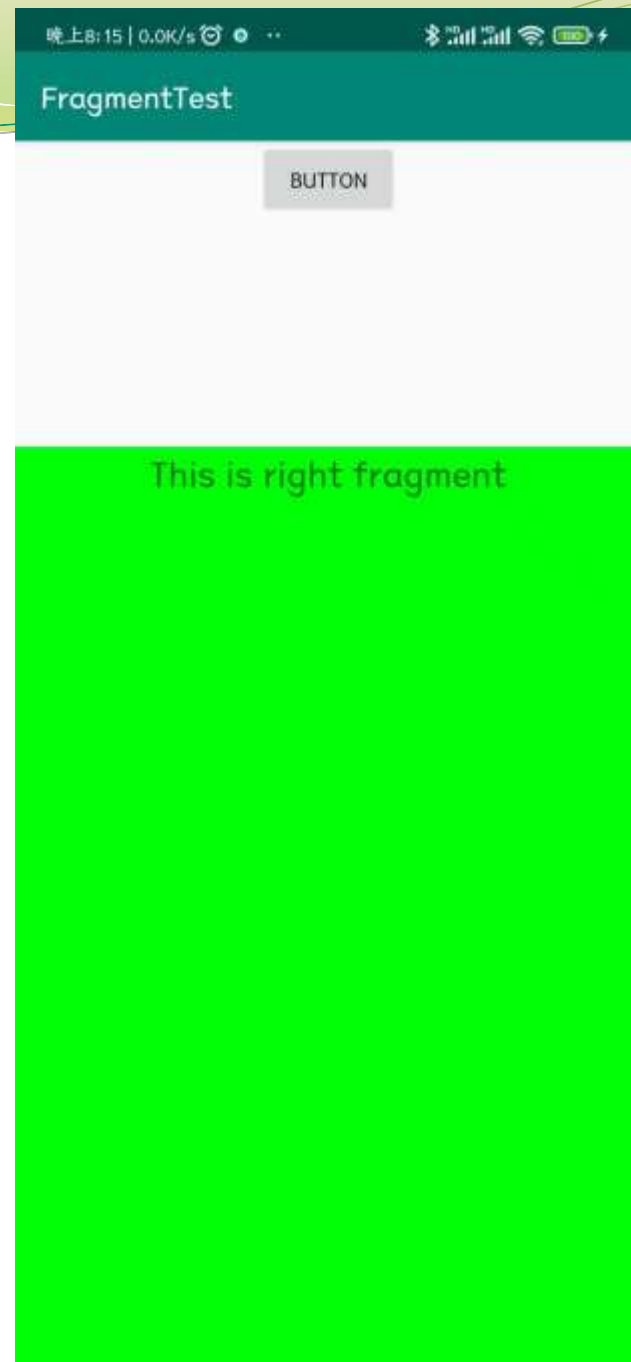
4

限定符的使用

使用限定符

在运行时可借助限定符（**qualifier**）来判断程序应该是使用双页模式还是单页模式

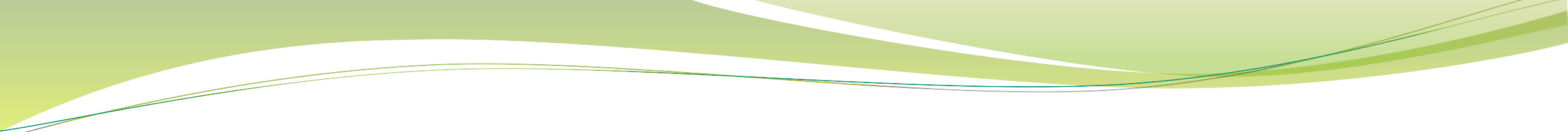
使用**large**限定符，那些屏幕被认为是**large**的设备就会自动加载**layout-large**文件夹下的布局，小屏幕的设备则还是会加载**layout**文件夹下的布局。



最小宽度限定符

使用最小宽度限定符（smallest-width qualifier）来更加灵活地为不同设备加载布局，不管它们是不是被系统认定为large。

举例：在res目录下新建layout-sw600dp文件夹，这样当程序运行在屏幕宽度大于等于600dp的设备上时，会加载layout-sw600dp/activity_main布局，当程序运行在屏幕宽度小于600dp的设备上时，则仍然加载默认的layout/activity_main布局。



结束

